Week 1: Formal & Informal Technical Reports

1 Sub-themes & Application: Core framework architectures, identifying infrastructure stakeholders, structural document integrity.

1.1 The Five Core Variables: System Mapping Before Writing

Before you start writing a technical report, lab manual, or project, thesis, or dissertation you must answer one master question: **"Who is talking to Whom about What, How, and Why?"** This question functions as an initialization script. If you leave out any of these variables, your document will fail to compile correctly for the reader. Here is exactly what each variable means in information technology:

1. **Why, The Purpose: Why Are You Writing?**
   Before you type a single word of a technical report, you must ask yourself one simple question: **"What do I want my reader to gain, behave, decide, think after reading the results of your writing?"** In information technology, we do not write just to fill pages. Every document is like a data packet sent over a network—it must have a clear destination and a specific job to do.
   There are three main types of purposesin technical writing:

   (a) **Purpose Type: To Write a Record (Saving System History)**
   What it means:
   This type of writing is used to keep an exact, step-by-step history of what happened to a system, server, or project so team members can check it later.

   (b) **IT Student Example:** The Server Outage Report (Post-Mortem Report)
   The Situation: A massive network attack causes the department database to crash, and the system goes offline for 4 hours.
   What you write: A clear, timed log showing exactly when the traffic spiked, which firewall settings failed, how you fixed it, and when the server came back online.
   How the reader behaves: Future network students and IT staff read this document like a history book. They use your record to learn from past mistakes and make sure the same server crash never happens again.

   (c) **Purpose Type: To Give or Request Information (Sharing Clear Technical Data) What it means:** This type of writing is used to share exact facts, instructions, code rules, or setup steps clearly without any confusion.

   (d) **IT Student Example:** API Reference Documentation
   The Situation: Your team is building a mobile app. The backend team creates a new login system, and the frontend team needs to know how to connect to it.
   What you write: A simple instruction document showing the exact URL path, the data format to send (like a username and password in JSON), and the success or error codes the server will return.
   How the reader behaves: The frontend programmers read your document to understand how the system works. They don't have to guess or waste time troubleshooting because your information gives them the exact data rules they need to write their code. **Purpose Type: To Influence (Persuading the Reader to Choose Your Solution) What it means:** This type of writing is used to convince a professor, a project panel, or a manager that your technical idea or choice is the best one.
   IT Student Example: The Capstone Project Proposal

Copyright © 2026 Dalya Raad AlAni

The Situation: You need to choose a cloud database for your final-year graduation project, and you want to use Firebase instead of a traditional MySQL database.
What you write: A project proposal that compares speed, setup costs, and ease of use, proving with facts and charts why Firebase is the better choice for your specific app.
How the reader behaves: The professors on the evaluation panel read your arguments. Because your document uses solid facts to persuade them, they decide to approve your project idea and give you permission to start building.

2. **"WHO": Establishing Your Technical Persona** In technical and academic writing, you do not write as an anonymous observer. You must explicitly define your User Persona before typing. Your technical role, your background experience, and your position in your organization act like an access token—they determine your credibility, your tone, and the scope of what you are allowed to declare. To figure out your "Who," you must answer three core questions before writing:

   (a) **What is your role?** (Are you a Creator designing a new app, a Maintainer documenting a server, or an Auditor reviewing security?)

   (b) **What is your experience baseline?** (Are you a student researcher learning a new framework, or a certified network admin?)

   (c) **What is your relationship to the reader?** (Are you writing up to an evaluation panel, out to a team of peer developers, or down to a non-technical end-user?)

   (a) **Example 1: The IT Security Auditor (The Critical Persona)**
   • **The Persona (Who):** You are an external Cybersecurity Consultant hired to perform a penetration test on a university's enrollment portal.
   • **Experience & Position:** You have deep expertise in network vulnerabilities, but you do not own the code. You are an objective third-party reviewer.
   • **How it affects the writing:** Your tone must be strictly objective, forensic, and highly critical. Because your role is to find faults, you write from a position of investigative authority. You do not hide flaws; you highlight them directly using strict vulnerability metrics (e.g., "The system currently exposes unencrypted user credentials via endpoint X...").

   (b) **Example 2: The Systems Designer / Capstone Student (The Justifying Persona)**
   • **The Persona (Who):** You are a Final-Year IT Bachelor Student writing the methodology chapter of your graduation thesis.
   • **Experience & Position:** You are a student creator. You have researched a new technology (like blockchain for supply chains) but are still being evaluated by a senior academic panel.
   • **How it affects the writing:** Your tone cannot be purely authoritative; it must be justifying and defensive. Since your position is subordinate to the professors reading it, your writing must explicitly prove how you gained your knowledge, acknowledge your project's limits, and cite established researchers to validate your design choices.

   (c) **Example 3: The DevOps Infrastructure Engineer (The Operational Persona)**
   • **The Persona (Who):** You are an Internal Cloud Administrator writing a server migration runbook for your peer deployment team.
   • **Experience & Position:** You are an experienced insider who knows the company's cloud architecture inside and out. You are writing to your equals (other engineers).
   • **How it affects the writing:** Your tone is highly practical, urgent, and compressed. There is no need to define basic concepts or defend your choices because your team already shares your experience level. You skip the theories and write direct, high-density instructions (e.g., "Run script v2.1 to migrate the Docker volume immediately after taking the local node offline").

Copyright © 2026 Dalya Raad AlAni

3. **"WHOM": Audience Mapping & Access Control** The "Whom" is the single most critical variable in technical communication. In information technology, writing without analyzing your audience is identical to broadcasting data packets blindly across a network with no IP address header—the data drops and causes a system-wide failure.
   To map your audience, you must classify them across two operational dimensions:

   (a) **Organizational Delta:** Are they above you (management/evaluators), equal to you (peer developers), or external (clients/users)?

   (b) **Audience Tier:** Who is your Primary Audience (the direct target who must act on the text) and who is your Secondary Audience (hidden stakeholders affected by the text)?

   (a) **Example 1:**
   • **The Audience (Whom):** The Academic Graduation Thesis Panel or a company's Chief Information Security Officer (CISO).
   • **Audience Classification:** Internal / Above You. They are the Primary Audience because they hold the authority to grade your capstone project or fund your department's infrastructure.
   • **How it affects the document:** You must write with maximum academic and structural rigor. You cannot skip theoretical steps or use casual language. If you are proposing a new encryption mechanism, you must include the full mathematical validation, complete system architecture blueprints, and extensive citations to prove your competence to those who outrank you.

   (b) **Example 2:**
   • **The Audience (Whom):** Your peer Backend Developers or Systems Administrators working on a shared GitHub repository.
   • **Audience Classification:** Internal / Equal Position.
   • **How it affects the document:** This document uses a high-density, low-explanation format. Since your peers share your exact technical background, you do not waste pages explaining what a "Docker container" or an "API endpoint" is. The document skips straight to the functional data payload: code snippets, configuration lines, and terminal strings (e.g., git clone setups) because your equals require immediate operational speed.

   (c) **Example 3:**
   • **The Audience (Whom):** A corporate client buying a custom database management tool, or the End-Users utilizing a campus registration network.
   • **Audience Classification:** External / Below or Outside your technical tier. They are the Primary Audience for user guides, but Secondary Audience stakeholders for technical system architecture files.
   • **How it affects the document:** You must implement a total technical filter. Jargon must be eliminated. If a non-technical customer reads an error troubleshooting guide, you do not show them raw SQL stack traces. Instead, you translate the technical breakdown into plain language and step-by-step layout visuals: "Click the blue 'Reset' button if the connection status light shifts from green to red."

4. **"WHAT": Defining Content Boundaries and Information Density** The "What" is the core payload of your document. It is the precise system configuration, dataset, or architectural concept you are communicating.
   Still, writing the "What" is not just about dumping information onto a page. The true skill of an IT professional lies in filtering: knowing exactly what to include and what to leave out. If you provide too much irrelevant data, your document suffers from cognitive overflow; if you provide

Copyright © 2026 Dalya Raad AlAni

too little, your system description fails its validation audit.

(a) **Example 1: Scoping a Database Migration Report**
• **The Scenario:** You are an IT student writing a laboratory report on migrating a local user management database to an AWS cloud environment.
• **The Main Message ("What"):** Proving that the data successfully moved to the cloud without corruption, and validating that the cloud database can handle real-time traffic queries.
• **The Data Dump (Poor Scoping):** Including 15 pages of raw, comma-separated SQL data logs showing every single user record that was copied. This causes massive text density and makes the professor skim past your actual analysis.
• **The Black Box (Poor Scoping):** Writing, "We ran a migration tool, and all profiles are now on Amazon servers." This lacks engineering precision and fails to prove success.
• **Perfect Scoping (Production-Grade):** You exclude the raw database logs. Instead, you include a high-level Entity-Relationship (ER) diagram of the schema, a concise summary table showing record counts before and after migration, and a single performance benchmark chart illustrating query response times.

(b) **Example 2:** Scoping a Network Security Incident Response Guide
• **The Scenario:** You are writing an urgent technical advisory notice to the university's internal staff regarding a new phishing vulnerability spreading through the campus Wi-Fi network.
• **The Main Message ("What"):** Informing employees how to spot the malicious email and how to safely report it to the IT helpdesk.
• **The Data Dump (Poor Scoping):** Including the full hexadecimal packet captures, the raw email SMTP routing headers, and an advanced breakdown of the malware's assembly code. The non-technical staff will become overwhelmed, panic, or ignore the warning entirely.
• **Perfect Scoping (Production-Grade):**
You completely exclude the network engineering logs. The "What" is strictly limited to three things: a screenshot of the fake email highlighting the sender's spoofed address, a 3-step bulleted verification list, and a direct hyperlink to the official IT ticketing portal.

5. **"CONTEXT": Analyzing the Deployment Environment**
The Context is the situational environment surrounding your document. It includes environmental factors like location, current system events, timing, and organization culture.
In information technology, you cannot configure software properly unless you know the environment where it will run (e.g., Development, Staging, or Production). Writing without checking the context is exactly like running a production script on a live server during peak user hours without checking for active system traffic—it causes a massive crash.

(a) **Example 1: The Context of Timing (Normal Status vs. Active System Crisis)**
• **The Scenario:** You need to update your team on a critical vulnerability found in the campus Wi-Fi authentication server.
• **Context Factor (Timing):**
o **Routine Context:** If discovered during summer break, you write a comprehensive, 10-page Security Architecture Refactor Proposal using LaTeX, outlining a complete code overhaul over the next two months.
o Crisis Context: If discovered on Monday morning during Final Exam Week, the context shifts completely. You do not write a long, theoretical paper. Instead, you send a rapid, high-impact Emergency Patch Advisory Alert via email or Slack, detailing a 2-step immediate firewall workaround to keep the servers online right now.

Copyright © 2026 Dalya Raad AlAni

(b) **Example 2: The Context of Location (Physical Environment & Constraints)**
• **The Scenario:** Writing a deployment manual for configuring network routers.
• **Context Factor (Location):**
o **Lab Context:** If your audience is IT students working in a comfortable campus computing lab with dual-monitor setups, your document can include a rich, multi-page layout with high-density system flowcharts and color-coded network schemas.
o **Field Context:** If your audience is a field technician standing on a ladder inside a cramped, dark, unventilated server closet with only a small smartphone screen to read the text, the context forces your document design to adapt. It must use single-column, large-font bulleted commands that can be read quickly on a mobile viewport without pinching and zooming.

(c) **Example 3: The Context of Organization Culture (Academic Rigor vs. Corporate Speed)**
• **The Scenario:** Documenting an open-source software library you developed for your final graduation project.
• **Context Factor (Culture):**
o **Academic Culture:** The context demands deep proof of concept. Your thesis must explicitly trace the mathematical algorithm, justify your data structures with peer-reviewed references, and defend your code logic to an evaluating panel.
o **Industry/Corporate Culture:** If you take that exact same project to an agile tech startup, the business culture does not value long academic papers. They demand an instant GitHub README.md file with a 1-line installation script (npm install) and an interactive setup example so devs can test it in under 30 seconds.

1.2 Introduction to Scientific Report Design (The IT Capstone Framework): Anatomy of an IT Graduation Thesis versus standard corporate system documentation. Understanding structural components, layout constraints, and macro technical document flow.

The Macro-Architecture of the IT Project—Compiling the IMRD Structure An academic document is not a compilation of personal opinions, nor is it an unstructured dump of code. In information technology, we write with structural precision. Every report we publish is identical to a complex data packet sent over a network. It must have a valid destination, specific data boundaries, and an explicit job to do. If you leave out any of the core variables we are discussing today, your document will fail to compile correctly for your reader, resulting in a rhetorical runtime error.
To prevent this system crash, academic writing relies on a globally validated schema known as the IMRD model. The IMRD model stands for Introduction, Methods, Results, and Discussion. In the context of an IT graduation thesis, this is not just an abstract layout; it is your macro technical document flow. It is the system pipeline through which your research payload travels. We will execute a complete structural audit of the IMRD architecture, mapped through the five core deployment variables of engineering communication: Why, Who, Whom, What, and Context.

1. **Introduction: Establishing the Network Territory and Initializing the Handshake**
The "I" in the IMRD model represents the Introduction. In software terms, the introduction functions as the initial network handshake protocol. You cannot simply begin a document by declaring what your code does; you must first establish a connection with the global computing landscape. Swales and Feak define this process through the CARS framework, which requires a writer to establish a research territory, identify a critical niche or structural gap, and then occupy that niche with their engineered solution.
When analyzing the variable of Why (The Purpose) within the introduction, your objective shifts

Copyright © 2026 Dalya Raad AlAni

from writing a simple system record to an active exercise in influence. Your purpose is to persuade an evaluating academic panel that a real-world computing vulnerability or architectural bottleneck exists, and that your project is the optimal patch required to fix it.
This directly alters your Who (The User Persona). In the introduction of an IT graduation thesis, you cannot write as a DevOps infrastructure engineer who drops a raw script without context. You must assume the persona of a Systems Designer. Because your academic position is subordinate to the evaluation panel reading your text, your tone must be justifying and defensive. You explicitly prove how you identified the technical problem, acknowledge the theoretical limits of your domain, and cite established protocols to validate your engineering direction.
The Whom (The Audience) for this introduction layer is ranked internally and sits above you in the organizational hierarchy. They require absolute structural and academic rigor. Therefore, the What (The Information Density) of your introduction must be perfectly scoped. You exclude low-level configuration lines and instead provide high-level conceptual boundaries: you define the cloud database paradigm, map out the existing system limitations, and explicitly state your research objectives. If you provide a raw data dump here, the evaluation panel will experience cognitive overflow and reject your document at the parsing phase.

2. **Methods: The Structural Blueprint and Abstract Class Implementation**
The ”M” in the IMRD model denotes the Methods section, which in computer engineering translates directly to your System Development Framework and Methodology. If the introduction is the handshake, the Methods section is the abstract class implementation of your architecture. It outlines the logical rules, database schemas, algorithmic constraints, and network topologies that govern your solution.
The Why (The Purpose) of the Methods section is to share clear, immutable technical data so that another engineer can clone your environment and replicate your exact results. It serves as a historical record of your engineering logic. Consequently, your Who (The User Persona) shifts from a defensive designer to an objective Systems Auditor and Maintainer. You write with clinical, forensic authority. You do not hide the architectural trade-offs or security limitations of your framework; you document them with absolute transparency using rigid engineering metrics.
Your Whom (The Audience) here includes your peer backend developers and systems administrators. Because they share your exact technical background, the What (The Information Density) shifts into a high-density, low-explanation layout. You completely eliminate conversational jargon and introductory definitions. You do not waste pages explaining what a Docker container or an API endpoint is. Instead, you scope the content strictly to production-grade design tools: high-level Entity-Relationship diagrams, detailed data-flow schematics, and algorithmic pseudocode blocks. The Context (The Deployment Environment) must also be clearly demarcated. You must explicitly define whether your framework was constructed within an isolated lab context using predictable dual-monitor simulation variables, or if it was built to adapt to a volatile field context where network packets drop due to real-world physical interference.

3. **Results: Telemetry Logs, Unit Tests, and Benchmark Validation**
The ”R” in the IMRD model signifies the Results. In the lifecycle of an IT project, the Results section is your production telemetry dashboard. It represents the actual empirical output generated when you compile your system and run your unit tests, load-testing frameworks, and stress scripts.
The primary Why (The Purpose) here is to give verifiable information. You are showing the exact performance history of your system under load. You are proving that your engineered artifact did not just compile in a vacuum, but successfully executed its target payload without corruption.
Your Who (The User Persona) during this phase is strictly data-driven and non-speculative. You do not comment on your feelings or make unsupported claims; you act as a recording sensor that outputs exact system state evaluations. The What (The Information Density) of the Results section requires the most disciplined application of filtering to avoid a catastrophic data dump. A poor IT student will copy and paste fifteen pages of raw, comma-separated SQL traffic logs

Copyright © 2026 Dalya Raad AlAni

or thousands of lines of terminal output showing standard debugging messages. This represents total scoping failure. A production-grade Results section completely excludes raw engineering logs. Instead, you translate that raw telemetry into concise summary tables showing data packet recovery counts before and after migration, and single performance benchmark charts that visually illustrate microsecond-level query response times.

Your Whom (The Audience) requires quick visual anchors to verify your system's efficiency. By presenting synthesized data charts instead of chaotic text strings, you honor the organizational delta of your readers, giving them the exact data rules they need to validate your software's performance metrics.

4. **Discussion: The Post-Mortem Report and System Refactoring**

The final component of the IMRD model is the "D", representing the Discussion. In standard corporate IT workflows, this operates exactly like an architectural Post-Mortem Report or a System Refactoring Analysis. This is where you zoom out from the isolated data numbers of your results and interpret what those metrics mean within the broader computing ecosystem.

The Why (The Purpose) of the Discussion is a hybrid of record-keeping and active influence. You are interpreting the unhandled exceptions of your design, analyzing why certain firewall configurations failed or why latency spiked under specific conditions, and persuading the reader that your overall architectural model remains a viable contribution to the IT field despite these constraints.

Your Who (The User Persona) returns to the Justifying Persona, but with an added layer of investigative authority. You directly confront the limitations of your project, comparing your benchmark outcomes against the baseline frameworks established by prior researchers in the Swalesian literature network.

The Context (The Deployment Environment) heavily dictates the formatting flow of this final macro component. If the document is deployed within an Academic Culture, the context demands deep proof-of-concept synthesis, where you justify your data anomalies using peer-reviewed computational theories. If this were standard corporate system documentation written for an agile startup culture, this entire phase would mutate into a short, compressed Markdown file containing future optimization tickets and an immediate feature roadmap for the next sprint. However, for your graduation thesis, you must provide the full macro technical flow, closing the loop of the IMRD handshake and proving that your system architecture is fully stable, authenticated, and ready for deployment.

Copyright © 2026 Dalya Raad AlAni

Dalya Raad Al-Ani

Digitally signed by Dalya Raad Al-Ani